Lab: Intermediate Code Generation

Lab tasks:

1. **Arithmetic Expression Intermediate Code Generation:**
   In this task, students will build a front-end of a compiler using Flex and Bison to parse arithmetic expressions containing variables, parentheses, and multiple operators (like `+`, `-`, `*`, `/`). The grammar must correctly handle operator precedence and associativity. Upon successful parsing, the tool should generate intermediate code in the form of Three-Address Code (TAC) using temporary variables. For example, the expression `a = (b + c) * (d - e) / f;` should result in a sequence of TAC instructions with intermediate results stored in `t1`, `t2`, etc.

2. **Intermediate Code Generation for If-Else Statements:**
   In this task, students will implement a simple compiler component that can parse `if-else` conditional statements with relational expressions, using Flex and Bison. The tool should generate intermediate code that includes conditional jumps and labels to represent control flow. For example, for the input `if (a < b) x = y + z; else x = y - z;`, the output should include appropriate labels (`L1`, `L2`) and branching using `ifFalse` statements. This task helps students understand how control structures are transformed into low-level representations during compilation.